HW2: Motion Retargeting

Method & formulas

This project transfers a captured walking motion from the source skeleton A to a different target skeleton B while preserving the original gait pattern and foot placement. The pipeline follows the assignment specification: quaternion math, forward kinematics (FK), inverse kinematics (CCD), and retargeting by joint-name mapping.

1. Quaternion math

Each joint stores a local rotation quaternion in the form (w, x, y, z). Quaternion operations are implemented using the standard Hamilton product:

- Normalize: $q = \dfrac{q}{\|q\|}$
- Multiply: $q_a q_b = (w, \mathbf{v})$ with
  - $w = w_a w_b - \mathbf{v}_a \cdot \mathbf{v}_b$
  - $\mathbf{v} = w_a \mathbf{v}_b + w_b \mathbf{v}_a + \mathbf{v}_a \times \mathbf{v}_b$
- Axis-angle rotation:
  - $q = \left(\cos\dfrac{\theta}{2},\ \mathbf{n}\sin\dfrac{\theta}{2}\right)$
  - with $\mathbf{n} = \dfrac{\mathbf{axis}}{\|\mathbf{axis}\|}$
- Vector rotation: $\mathbf{v}' = q\,\mathbf{v}\,q^{-1}$

The composition convention is important: $a * b$ applies b first and then a. The identity quaternion is $(1, 0, 0, 0)$.

2. Forward kinematics

The skeleton is a hierarchy stored in parent-before-child order. For a root joint:

- $P_0 = \mathrm{rootTranslation} + \mathrm{offset}_0$
- $Q_0 = \mathrm{normalize}(\mathrm{localRotation}_0)$

For any child joint j with parent p:

- $P_j = P_p + \text{rotate}(Q_p,\ \text{offset}_j)$
- $Q_j = \text{normalize}(Q_p \cdot \text{localRotation}_j)$

This ensures each bone is placed in world space using the parent transform and then oriented by the parent rotation followed by the local joint rotation.

Per-frame pipeline and IK goals. For each frame f, the program (1) runs source FK to obtain world positions; (2) calls retargetFrame with rootScale = 1.12 to obtain the target pose; (3) computes a world-space goal for each target foot; (4) calls solveCCD once for the left foot and once for the right foot, each with at most 30 passes; and (5) runs FK on the corrected target pose for display.

The goal is the source foot's position relative to the source root, scaled by 1.12 and re-attached to the target root:

$$r_A = P_A[0](sourcerootworldposition)$$
$$r_B = rootTranslation_B + offset_B[0]$$
$$goal_foot = r_B + 1.12 \cdot (P_A[foot] - r_A)$$

The left and right goals use LeftFoot and RightFoot of A, and the solver moves B_LeftFoot and B_RightFoot. Because the goal is expressed relative to the root, the IK keeps each foot at the same relative placement as in the source (scaled), instead of keeping the foot at the same absolute world position. The two legs are solved independently, since they do not share any joint below the root.

3. CCD inverse kinematics

The left and right feet are solved independently with cyclic coordinate descent. For a foot target $\mathbf{t}$ in world coordinates, we choose the leg chain from the lower-leg parent to the upper-leg parent and iterate until the end effector reaches the target within $1\ \text{cm}$ or a maximum of 30 iterations.

At each joint update, let:

- $\mathbf{pivot}$ = world position of the current joint
- $\mathbf{foot}$ = world position of the end effector
- $\mathbf{current} = \mathbf{foot} - \mathbf{pivot}$
- $\mathbf{desired} = \mathbf{t} - \mathbf{pivot}$

The alignment rotation is computed in world space using the cross and dot products:

- $\mathbf{axis} = \text{normalize}(\mathbf{current} \times \mathbf{desired})$
- $\text{angle} = \arccos\Big(\ \text{clamp}\ \big(\ \text{normalize}(\mathbf{current}) \cdot \text{normalize}(\mathbf{desired}),\ -1,\ 1\big)\Big)$

The world correction is then converted to the local frame of the adjusted joint:

- $\text{localDelta} = \text{parentWorld}^{-1} \cdot \text{worldDelta} \cdot \text{parentWorld}$
- $\text{newLocalRotation} = \text{normalize}(\text{localDelta} \cdot \text{oldLocalRotation})$

Recomputing FK after each joint update keeps the chain consistent and allows the foot to converge toward the target while respecting the hierarchy.

4. Retargeting

The target rig uses the same joint hierarchy but with naming prefixed by B_, and the limb lengths differ from the source rig. Retargeting therefore does not assume matching indices. Instead, each target joint B_ is mapped to the source joint by name; the source local rotation is copied and normalized into the target pose. The root translation is scaled by 1.12 to account for the target body proportions, while each target joint retains its own bone offset so the new limb lengths are respected. Unmatched names throw a clear error so the pipeline fails early instead of silently producing wrong motion.

Implementation decisions

Several design choices were necessary to make the motion transfer robust and consistent with the assignment specification:

1. Quaternion math

```cpp
Quat normalize(Quat q) {
    // TODO 1a: return q divided by its length. Handle a near-zero quaternion.
    const double norm = std::hypot(std::hypot(q.w, q.x), std::hypot(q.y, q.z));
    if (norm <= 1e-12) return {};
    return {q.w / norm, q.x / norm, q.y / norm, q.z / norm};
}

Quat multiply(Quat a, Quat b) {
    // TODO 1b: Hamilton product. Composition convention: a*b applies b, then a.
    double a_w = a.w * b.w - a.x * b.x - a.y * b.y - a.z * b.z;
    double a_x = a.w * b.x + a.x * b.w + a.y * b.z - a.z * b.y;
    double a_y = a.w * b.y - a.x * b.z + a.y * b.w + a.z * b.x;
    double a_z = a.w * b.z + a.x * b.y - a.y * b.x + a.z * b.w;
    a.w = a_w; a.x = a_x; a.y = a_y; a.z = a_z;
    return a;
}
```

`normalize` computes the quaternion's Euclidean length and divides each component by it, producing a unit quaternion for rotation composition. To avoid division by a near-zero length, it returns the default quaternion (the identity) when the length is at most 10^{-12}. `multiply` implements the Hamilton

product, combining the scalar and vector components; the order is significant, with $a * b$ applying b first and then a.

```cpp
Quat axisAngle(Vec3 axis, double radians) {
    // TODO 1c: normalize axis, use cos(theta/2) and sin(theta/2).
    const double axis_length = length(axis);
    Vec3 normalized_axis{};
    if (axis_length > 1e-12) {
        normalized_axis = {axis.x / axis_length, axis.y / axis_length, axis.z / axis_length};
    }
    double half_angle = radians / 2.0;
    double sin_half_angle = std::sin(half_angle);
    double cos_half_angle = std::cos(half_angle);
    // Return quaternion in the form (w, x, y, z)
    return {cos_half_angle, normalized_axis.x * sin_half_angle, normalized_axis.y * sin_half_angle, normalized_axis.z * sin_half_angle};
}

Vec3 rotate(Quat q, Vec3 v) {
    // TODO 1d: use q*(0,v)*conjugate(q), or an equivalent stable formula.
    // Convert vector to quaternion with w=0
    Quat v_quat{0, v.x, v.y, v.z};
    // Calculate the conjugate of q
    Quat q_conjugate{q.w, -q.x, -q.y, -q.z};
    // Perform the rotation: q * v_quat * q_conjugate
    Quat temp = multiply(q, v_quat);
    Quat rotated_quat = multiply(temp, q_conjugate);
    // Extract the rotated vector from the resulting quaternion
    v.x = rotated_quat.x;
    v.y = rotated_quat.y;
    v.z = rotated_quat.z;
    return v;
}
```

`axisAngle` first normalizes the rotation axis, then forms $(\cos(\theta/2),\ \mathbf{n}\sin(\theta/2))$, where θ is in radians. `rotate` treats the vector as the pure quaternion $(0, \mathbf{v})$ and computes $q(0, \mathbf{v})\bar{q}$, where $\bar{q}$ is the conjugate of q; the resulting vector components are the rotated vector. This rotation formula assumes q is a unit quaternion, so rotations are normalized when needed before use.

2. Forward kinematics

```cpp
WorldPose forwardKinematics(const Skeleton& skeleton, const Pose& pose) {
    WorldPose world;
    world.position.resize(skeleton.joints.size());
    world.rotation.resize(skeleton.joints.size());
    // TODO 2: traverse joints in parent-before-child order. For a root,
    // position = rootTranslation + offset; rotation = localRotation.
    // For a child: position = parentPosition + rotate(parentRotation, offset)
    // and rotation = normalize(parentRotation * localRotation).
    // The lines below only show both rigs in their rest shape.
    for (size_t i = 0; i < skeleton.joints.size(); ++i) {
        const auto& j = skeleton.joints[i];
        world.position[i] = j.parent < 0 ? pose.rootTranslation + j.offset
                                         : world.position[j.parent] + rotate(world.rotation[j.parent], j.offset);
        world.rotation[i] = j.parent < 0 ? normalize(pose.localRotation[i])
                                         : normalize(multiply(world.rotation[j.parent], pose.localRotation[i]));
    }
    return world;
}
```

`forwardKinematics` walks joints in parent-before-child order and stores each joint's world-space position and rotation. A root is placed at `rootTranslation + offset` and uses its normalized local

rotation. For a child, the parent world rotation transforms the child's offset before it is added to the parent position; the child's world rotation is the normalized product of the parent world rotation and its local rotation.

3. CCD inverse kinematics

```
Vec3 cross(Vec3 a, Vec3 b) {
    return {a.y*b.z - a.z*b.y, a.z*b.x - a.x*b.z, a.x*b.y - a.y*b.x};
}

Vec3 anyPerpendicular(Vec3 v) {            // v 已正規化
    const Vec3 basis = std::abs(v.x) <= std::abs(v.y) && std::abs(v.x) <= std::abs(v.z)
        ? Vec3{1,0,0} : (std::abs(v.y) <= std::abs(v.z) ? Vec3{0,1,0} : Vec3{0,0,1});
    return cross(v, basis);
}

void rotateJointToward(const Skeleton& sk, Pose& pose, int joint, int effector, Vec3 target) {
    const WorldPose w = forwardKinematics(sk, pose);
    const Vec3 pivot = w.position[joint];
    Vec3 cur = w.position[effector] - pivot;
    Vec3 des = target - pivot;
    const double cl = length(cur), dl = length(des);
    if (!(cl > 1e-9 && dl > 1e-9)) return;   // 同時擋掉 NaN 與零向量
    cur = cur * (1.0 / cl);
    des = des * (1.0 / dl);

    const double c = std::clamp(dot(cur, des), -1.0, 1.0);
    Vec3 axis = cross(cur, des);
    if (length(axis) < 1e-12) {
        if (c >= 0.0) return;                // 已對齊
        axis = anyPerpendicular(cur);        // 反向
    }

    const Quat worldDelta = axisAngle(axis, std::acos(c));   // axisAngle 會自己正規化軸
    const Quat pw = w.rotation[sk.joints[joint].parent];     // FK 已 normalize
    const Quat inv{pw.w, -pw.x, -pw.y, -pw.z};
    const Quat localDelta = multiply(multiply(inv, worldDelta), pw);
    pose.localRotation[joint] = normalize(multiply(localDelta, pose.localRotation[joint]));
}
} // namespace
```

For each adjustable joint, the solver uses forward kinematics to find the pivot, current end-effector direction, and target direction in world space. It normalizes the directions, clamps their dot product to $[-1, 1]$, and uses the cross product as the correction axis and $\arccos$ of the clamped dot product as the angle. If the directions are already aligned, no update is needed; if they point in opposite directions, a perpendicular axis is chosen so a 180-degree correction can still be formed. Zero-length or invalid direction vectors are skipped.

```cpp
void solveCCD(const Skeleton& sk, Pose& pose, int ee, Vec3 target, int iterations) {
    const int n = static_cast<int>(sk.joints.size());
    if (iterations <= 0 || ee < 0 || ee >= n || pose.localRotation.size() != sk.joints.size()) return;

    const int j1 = sk.joints[ee].parent;
    if (j1 < 0) return;
    const int chain[] = {j1, sk.joints[j1].parent};
    constexpr double tol = 0.01;

    for (int it = 0; it < iterations; ++it) {
        for (int joint : chain) {
            if (length(target - forwardKinematics(sk, pose).position[ee]) < tol) return;
            if (joint < 0 || sk.joints[joint].parent < 0) continue;   // 不轉 root
            rotateJointToward(sk, pose, joint, ee, target);
        }
        if (length(target - forwardKinematics(sk, pose).position[ee]) < tol) return;
    }
}
```

`solveCCD` adjusts the end effector's parent and then that joint's parent, skipping the root so the body's root transform is not changed. Before and after updates it recomputes FK and stops once the end effector is within $1\ \mathrm{cm}$ of the target, or when the requested iteration limit is reached. Each world-space correction is conjugated by the parent world rotation to express it in the joint's local frame, then multiplied into and normalized with the joint's existing local rotation. FK is recomputed for each joint update so subsequent corrections use the updated pose.

4. Retargeting

```cpp
Pose retargetFrame(const Skeleton& source, const Skeleton& target,
                   const Pose& sourcePose, double rootScale) {
    // TODO 4: map B_<source-name> to each target joint, transfer local
    // quaternion motion, and normalize. Do not use same-index assumptions.
    // Report unmatched joints clearly. Distinct offsets in target.rig supply
    // different bone lengths. Root translation is scaled by rootScale.
    Pose result;
    result.rootTranslation = sourcePose.rootTranslation * rootScale;
    result.localRotation.resize(target.joints.size());

    std::unordered_map<std::string, std::size_t> sourceJointIndices;
    for (std::size_t i = 0; i < source.joints.size(); ++i) {
        sourceJointIndices[source.joints[i].name] = i;
    }

    for (std::size_t i = 0; i < target.joints.size(); ++i) {
        const std::string& targetName = target.joints[i].name;
        if (targetName.rfind("B_", 0) != 0) {
            throw std::runtime_error(
                "Target joint '" + targetName + "' must start with 'B_' to map to a source joint");
        }

        const std::string sourceName = targetName.substr(2);
        const auto sourceJoint = sourceJointIndices.find(sourceName);
        if (sourceJoint == sourceJointIndices.end()) {
            throw std::runtime_error(
                "No source joint '" + sourceName + "' matches target joint '" + targetName + "'");
        }
        if (sourceJoint->second >= sourcePose.localRotation.size()) {
            throw std::runtime_error(
                "Source pose has no rotation for joint '" + sourceName +
                "' required by target joint '" + targetName + "'");
        }

        result.localRotation[i] = normalize(sourcePose.localRotation[sourceJoint->second]);
    }

    return result;
}
```

`retargetFrame` scales the source root translation by the supplied `rootScale` and builds a lookup from source joint names to their indices. For every target joint, it removes the required `B_` prefix, finds the corresponding source joint by name, and copies that source pose's local rotation after normalization; it does not rely on the two skeletons having matching joint indices. The target skeleton's own offsets are unchanged, so its bone lengths are preserved. Missing prefixes, unmatched source joints, or missing source-pose rotations produce explicit errors.

FK/IK checks

The implementation was validated through the following checks:

- FK checks: a rotated parent should move its child in world space according to the parent rotation, and the root joint should remain at rootTranslation + offset[0].

```
// 檢查一:rest pose
CHECK(nearV(w.position[0], {0,1,0}),   "rest: root");
CHECK(nearV(w.position[1], {0,1.5,0}), "rest: spine");
CHECK(nearV(w.position[2], {1,1.5,0}), "rest: arm");

// 檢查二:旋轉父關節
p = identityPose(s);
p.localRotation[1] = axisAngle({0,0,1}, PI/2);
w = forwardKinematics(s, p);
CHECK(nearV(w.position[1], {0,1.5,0}), "rot spine: spine itself stays");
CHECK(nearV(w.position[2], {0,2.5,0}), "rot spine: arm moves to (0,2.5,0)");
// root 平移
p = identityPose(s);
p.rootTranslation = {1, 2, 3};
w = forwardKinematics(s, p);
CHECK(nearV(w.position[0], {1,3,3}),     "translate: root");
CHECK(nearV(w.position[2], {2,3.5,3}),   "translate: arm");
CHECK(nearV(w.position[1], {1,3.5,3}), "translate: spine");
```

The test uses a three-joint chain: the root offset is $(0, 1, 0)$, the spine offset is $(0, 0.5, 0)$, and the arm offset is $(1, 0, 0)$. In the rest pose, these offsets place the root, spine, and arm at $(0, 1, 0)$, $(0, 1.5, 0)$, and $(1, 1.5, 0)$ respectively. Rotating the spine by 90° around the Z axis leaves the spine's own position unchanged, but rotates its child's offset from $(1, 0, 0)$ to $(0, 1, 0)$; consequently, the arm moves to $(0, 2.5, 0)$. The final checks translate the root by $(1, 2, 3)$ and confirm that the root becomes $(1, 3, 3)$ while the spine and arm positions are translated by the same amount.

```
> ./build/fk_check
ok:   rest: root
ok:   rest: spine
ok:   rest: arm
ok:   rot spine: spine itself stays
ok:   rot spine: arm moves to (0,2.5,0)
ok:   translate: root
ok:   translate: arm
ok:   translate: spine
```

Every assertion shown in the test output passes, and the program reports zero failures. The test also checks rotation composition: with a 90° root rotation around Y and a 90° spine rotation around X, the arm reaches $(0, 1.5, -1)$. This verifies the parent-first world-rotation order, $Q_j = Q_p Q_{\text{local},j}$, in addition to the position checks shown above.

- IK checks: the CCD test covers reachable and unreachable targets, degenerate directions, and joints that should remain unchanged.

tests / ik_check.cpp - IK validation cases

```cpp
static Skeleton makeLeg(){
Skeleton s;
s.joints.push_back({"Hips",-1,{0,1.0,0}});
s.joints.push_back({"UpLeg",0,{0.1,0,0}});
s.joints.push_back({"Leg",1,{0,-0.45,0}});
s.joints.push_back({"Foot",2,{0,-0.42,0}});
return s;
}
static Pose identityPose(const Skeleton& s){
Pose p; p.localRotation.assign(s.joints.size(), Quat{1,0,0,0}); return p;
}
static Vec3 footPos(const Skeleton& s,const Pose& p,int foot){
return forwardKinematics(s,p).position[foot];
}
int main(){
Skeleton sk = makeLeg(); const int FOOT=3, HIP=1;
int fails=0;
std::ofstream csv("ik_convergence.csv"); csv<<"case,iter,error_m\n";
Pose start = identityPose(sk);
Pose ref = start;
ref.localRotation[1] = axisAngle({1,0,0}, 0.5);
ref.localRotation[2] = axisAngle({1,0,0}, -0.7);
Vec3 target = footPos(sk, ref, FOOT);
double before = dist(footPos(sk,start,FOOT), target);
Pose out = start; solveCCD(sk,out,FOOT,target,30);
double after = dist(footPos(sk,out,FOOT), target);
std::printf("[reachable] before=%.4f m after=%.5f m\n",before,after);
if(!(after<0.01)) {std::puts("FAIL: not within 1 cm"); ++fails;}
for(int it=0; it<=30; ++it){
Pose p = start; solveCCD(sk,p,FOOT,target,it);
csv<<"reachable,"<<it<<","<<dist(footPos(sk,p,FOOT),target)<<"\n";
}
Vec3 far = footPos(sk,start,FOOT) + Vec3{0,0,3.0};
Pose pf = start; solveCCD(sk,pf,FOOT,far,30);
WorldPose w = forwardKinematics(sk,pf);
bool ok = true;
for(auto& q: pf.localRotation) ok = ok && finiteQ(q);
for(auto& v: w.position) ok = ok && std::isfinite(v.x)&&std::isfinite(v.y)&&std::isfinite(v.z);
double d0 = dist(footPos(sk,start,FOOT), far), d1 = dist(w.position[FOOT], far);
std::printf("[unreachable] finite=%d before=%.4f after=%.4f\n",ok,d0,d1);
if(!ok || !(d1<=d0+1e-9)) {std::puts("FAIL: unreachable case"); ++fails;}
Pose p2=start; solveCCD(sk,p2,FOOT,footPos(sk,start,FOOT),30);
if(dist(footPos(sk,p2,FOOT),footPos(sk,start,FOOT))>1e-6){std::puts("FAIL: already-at-goal moved");++fails;}
Pose p3=start; solveCCD(sk,p3,FOOT,w.position[HIP],30);
for(auto& q:p3.localRotation) if(!finiteQ(q)){std::puts("FAIL: NaN at zero-length");++fails;}
if(out.localRotation[0].w!=1||out.localRotation[3].w!=1){std::puts("FAIL: hip/foot modified");++fails;}
std::puts(fails? "IK CHECK FAILED":"IK CHECK PASSED");
return fails?1:0;
```

The test builds a four-joint leg and creates a reachable target from a known pose. Starting from the identity pose, the foot is initially $0.1467\,\text{m}$ from the target; after 30 CCD iterations, the error is $0.00954\,\text{m}$, within the $0.01\,\text{m}$ tolerance. It then places the target $3\,\text{m}$ away from the initial foot position, beyond the leg's reach. The solver cannot reach this target, but the result remains finite and the error does not increase (from $3.0000\,\text{m}$ to $2.2536\,\text{m}$). Additional assertions check that an already-satisfied target does not move the foot, a zero-length direction toward the UpLeg pivot does not produce invalid rotations, and the hip and foot quaternion scalar components remain at their identity values.

```
Terminal - IK test execution

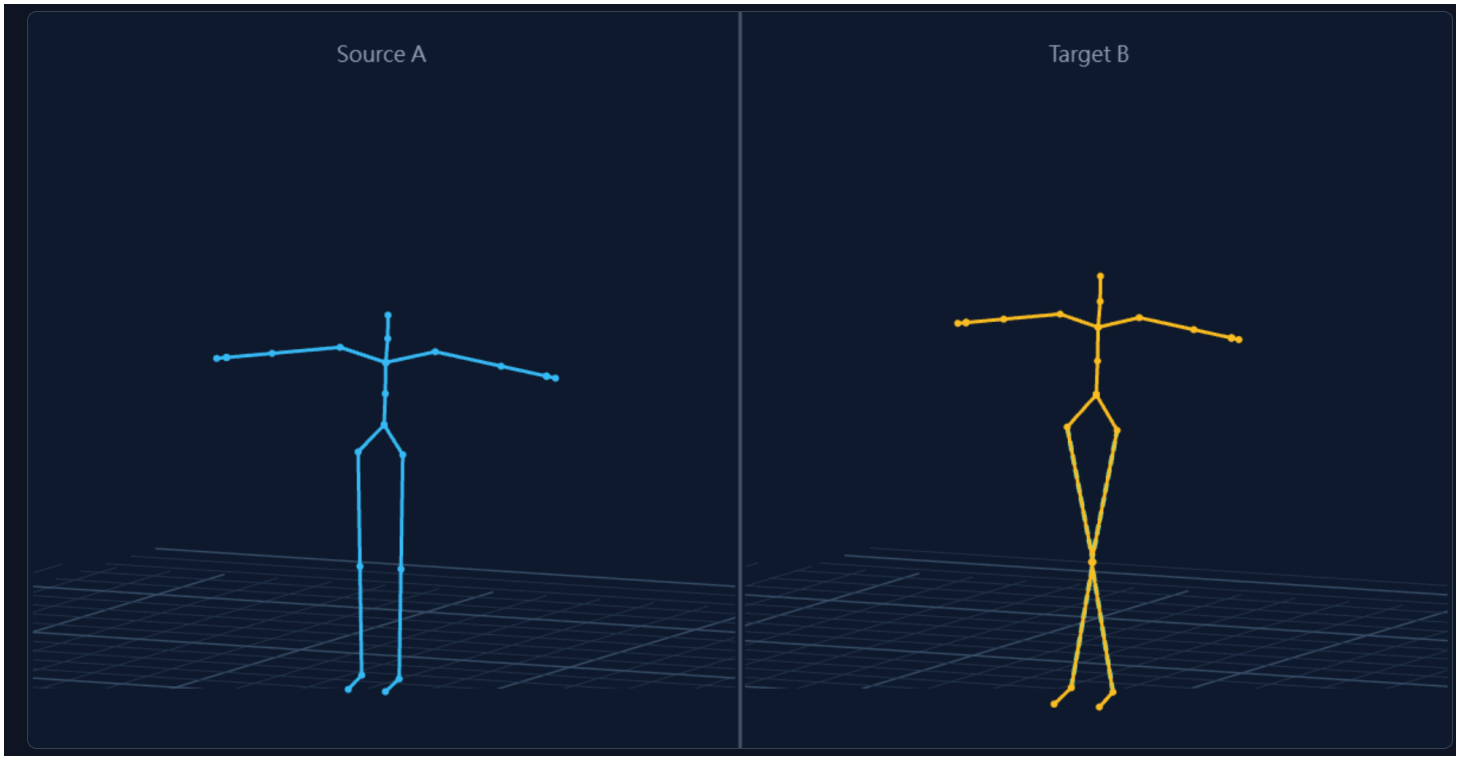
PS D:\Motion-Retargeting> wsl -d Ubuntu-22.04 -- bash -lc "cd /mnt/d/Motion-Retargeting && ./build/ik_check"
[reachable] before=0.1467 m after=0.00954 m
[unreachable] finite=1 before=3.0000 after=2.2536
IK CHECK PASSED
```

The test output reports the measured errors for the reachable and unreachable cases and ends with `IK CHECK PASSED`. That final status means all assertions passed, including the robustness and unchanged-joint checks, which print failure messages only if they fail.

In short, the FK results match the hierarchical transformation equations, and the CCD solver reaches each planted foot target while keeping the update local to the leg chain.

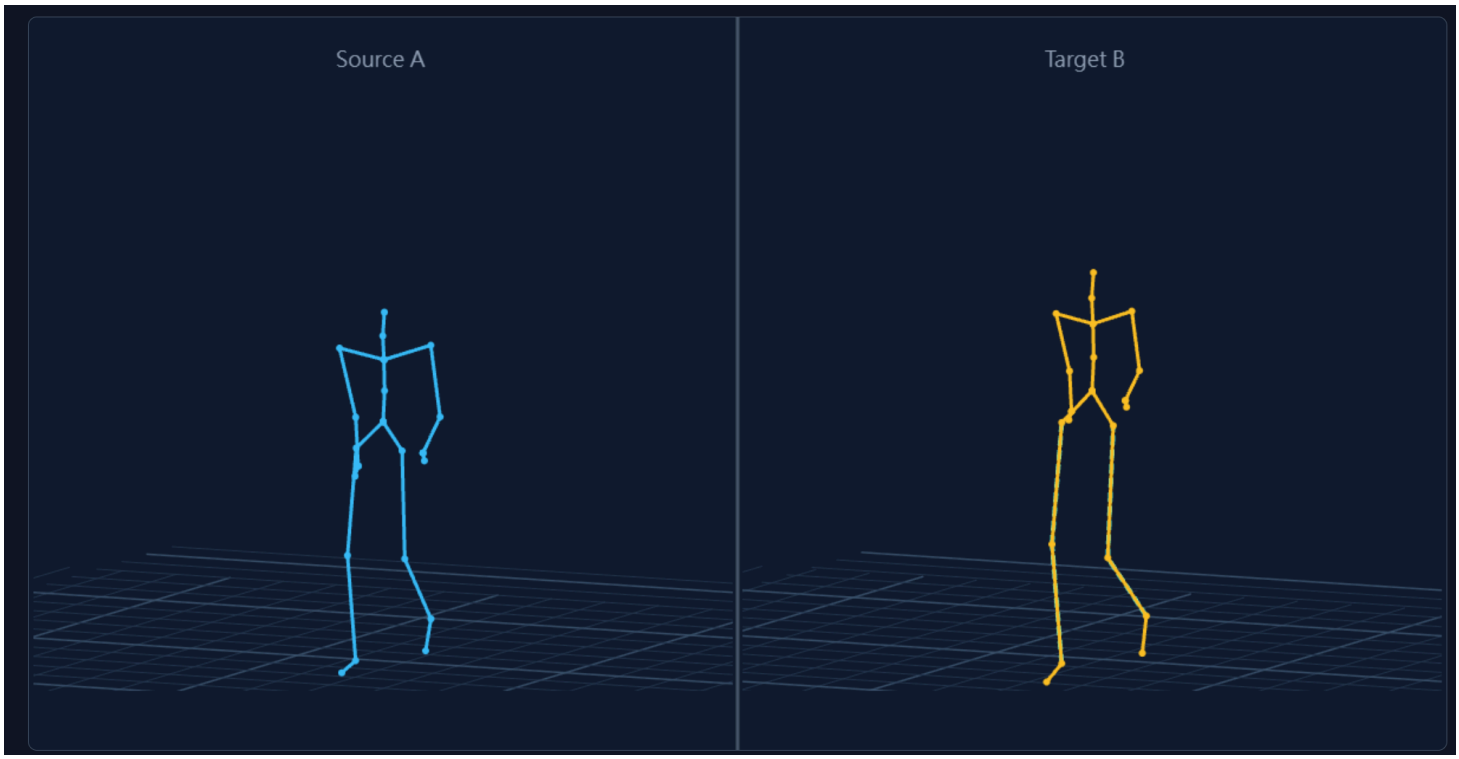
Comparisons

The source and target rigs differ in both bone lengths and overall body scale. The source motion is a CMU walk captured from subject 02, trial 01; the target rig B is created specifically for the assignment and has the same parent structure but different limb and torso proportions. Therefore, the retargeting process is not a direct copy of the source pose but a semantic transfer of local rotations onto the target skeleton.



Compared with a naive same-index copy, the name-based mapping keeps the motion consistent with the rig's joint semantics. Because B retains its own offsets, the transferred motion appears as the

same walking cycle but adjusted to the target proportions. The root translation also changes by the scaling factor 1.12, producing a larger overall target body while preserving the original stride rhythm and foot contact timing.



The viewer result should show a valid gait cycle with the same general movement pattern but with lengthened or shortened limbs depending on the target rig. The differences are most visible in the leg segments and torso spacing, while the forward motion and periodic foot placement remain recognizably from the source clip.

Limitations

The current method is effective for a simple motion retargeting task, but it has several limitations:

- The method does not enforce joint-angle limits, so the regenerated motion can overshoot or violate anatomical constraints.
- CCD only solves the feet using a two-joint leg chain; it does not consider full-body balance, pelvis drift, or global support constraints.
- Retargeting preserves joint rotations but cannot perfectly account for differences in bone lengths and the center of mass without a full inverse-kinematic or optimization-based correction.
- The solver is local and may converge to a poor local minimum when the goal is unreachable or heavily angled.
- The assignment version uses the provided source and target rigs only; broader character retargeting would require more complete rig correspondence and contact handling.

These limitations are acceptable for the homework setting, but they also explain why a more production-ready retargeting system would require stronger constraints, better foot-locking logic, and more advanced motion optimization.